Tool calling

Why models need tools

A language model only produces text. It cannot read a clock, query a database or do reliable arithmetic by itself. Ask it for today's date and it can only guess from patterns in its training data, which ended months or years before you asked. Tool calling lets it ask your program to do those jobs for it, so the answer comes from the real world and not from a guess.

A tool is simply a function in your program that you choose to offer: get the time, add two numbers, search a folder of documents, send an email. The model never sees the function's code. It sees only a short description of what the tool is for and what arguments it takes.

Describing a tool

You describe each tool with a name, a plain-English description and a schema for its arguments. The name is how the model refers to the tool. The schema lists each argument, its type, and which ones are required, usually written in a format called JSON Schema. Providers use slightly different wrappers around this, but the contents are the same.

The description matters most, because it is how the model decides whether a tool fits the question. A description such as "Adds two numbers and returns the sum" is clear. A vague one such as "Math helper" leaves the model guessing, and it will either skip the tool when it should use it or reach for it when it should not. Write the description as if you were explaining the tool to a new colleague who cannot read the code: say what it does, when to use it, and what it returns.

Keep the list of tools short and the tools distinct. If two descriptions overlap, the model has to choose between them, and it will sometimes choose wrongly. A few sharp tools work better than many fuzzy ones.

Who runs the tool

The model never runs a tool. It only replies with the tool name and the arguments, and your code decides whether and how to run it. This is the most important fact about tool calling, and it is easy to get backwards because the wording suggests the model "uses" the tool.

What really happens is a request and a response. The model writes something like "call add with a equal to 17 and b equal to 25". Your program reads that, looks up the function with that name, checks the arguments, and runs it. Your code is in charge the whole time. It can refuse a request, ask the user to confirm it, limit what the tool can touch, or log every call.

This is also why tool calling is safe enough to use. A tool that deletes files should check the path before it deletes anything, because the model might ask for a path it should not. Treat the arguments from a model the way you would treat input from a stranger on the internet.

Sending the result back

After your code runs the tool, the result goes back to the model as a new message tagged with the id of the request it answers. Every tool request carries an id, and the result must carry the same one. This is how the model matches answers to requests when it asks for several tools in one reply. If the ids are missing or wrong, the provider rejects the next request with an error.

The result is usually text. If the tool returns a number or a structure, convert it to a string first, such as JSON. Keep results small and relevant, because they are added to the conversation and the model must read them on every later round.

Then the program calls the model again with the longer conversation. The model reads the result and either writes its final answer or asks for another tool.

Arguments can be wrong

Models are good at filling in arguments but they are not perfect. They may leave out a required argument, pass a number as a word, or invent a name that does not exist. Your code should validate before it runs anything. If an argument is bad, do not crash: return a short error message as the tool result, such as "argument b must be a number". The model usually reads the message and tries again with a fix.

The same advice covers a tool name that does not exist. Return "unknown tool" as the result and let the model recover.

Choosing what to expose

Every tool you offer is a capability you have to defend. Start with the smallest set that does the job. Prefer read-only tools to ones that change data. When a tool must change something, make it narrow: a tool that can add a calendar event is much safer than a tool that can run any shell command.

Each tool description also takes up room in the prompt, so a lengthy list costs money on every request. A small, well-described set of tools is cheaper, easier to test and easier for the model to use correctly.

Summary

Tool calling is a simple contract. You describe tools in words and schemas. The model replies with a tool name and arguments when a tool fits. Your program runs the tool, returns the result tagged with the request id, and calls the model again. The model never executes anything itself, and that gap between asking and doing is where you put your

checks, your limits and your logs.